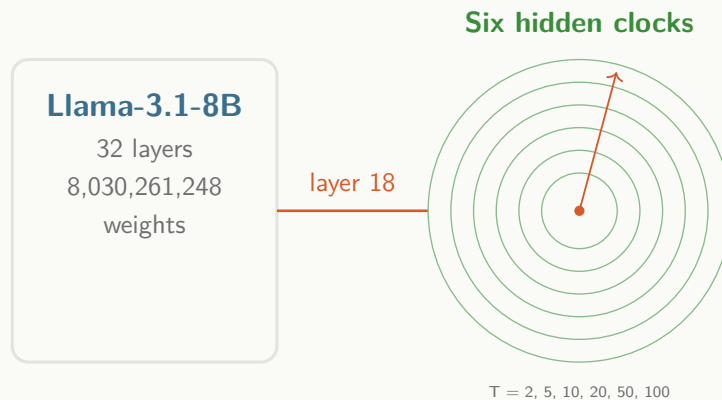


The Hidden Calculator Inside a Giant AI Brain

A Sequel: How a 200-Year-Old French Math Idea
Helped Decode a ChatGPT-Sized Model

Notes for curious students who haven't met advanced math yet

2026



This is the sequel to “The Secret Geometry of a Tiny AI Brain”. In the first book we cracked open a network with 4.8 million weights and found twelve glowing circles — the Fourier algorithm by which the brain divided numbers on a 97-hour clock. Beautiful, but: that brain was tiny, and it only did one task.

The real question always was: do giant AI brains — ChatGPT, Claude, Llama, the models with billions of weights that talk to you about anything — also use hidden clocks? Or do they cheat by memorizing?

In 2026 a team at a company called Goodfire answered this question. They invented a special tool, pointed it at Llama-3.1-8B (an open model with 8 billion weights), and discovered *six* hidden clocks running at layer 18, doing arithmetic for the entire model. Worse — or better, depending on your taste — those same six clocks light up when Llama reasons about *weekdays*, *months*, and *clock times*. One calculator, six clocks, every counting task in the model.

This book tells you how they found it, how it works, how we repeated their experiment ourselves last week, and what we discovered when we did. As before: no calculus, no trigonometry, just curiosity and a willingness to think about clocks.

Contents

Welcome Back, Detective	3
I From Tiny Brain to Giant Brain	4
1 Meet the Giant	5
2 Why You Can't Just Look Inside	7
3 The Probe — A One-Question Detector	8
4 Why a Clock?	10
II The Calculator Hiding in Plain Sight	12
5 Six Clocks at Layer Eighteen	13
6 Why Six? The Number-Identification Game	14
7 The Astonishing Coincidence	16
III Steering — Using the Clocks Against the AI	18
8 Rewinding the Clock	19
9 Making Llama Say September Instead of November	22
IV Our Detective Work	23
10 Five Failures and a Replication	24
11 The Numbers We Got	26
12 Why This Matters	29
13 What Comes Next in Our Lab	31
V After the Replication	32
14 One Calculator, or Many?	33

15 Five Ways to Steer a Giant Brain	36
Closing Thoughts	40

Welcome Back, Detective

If you read the first book, you already know:

- what a *neural network* is (a long line of helpers passing notes),
- what *modular arithmetic* is (math on a clock with N positions instead of 12),
- what the *Fourier basis* is (a way to describe any wiggly pattern as a sum of pure rotations),
- and most importantly, that a tiny network, trained on a *single* task — division on a 97-hour clock — internally stored its solution as twelve hidden circles rotating at twelve different speeds. That’s a Fourier calculator.

If you didn’t read the first book: you can still follow this one. We’ll re-explain the bits we need.

What’s new in this book: **the brain is enormous**. The kind that can write essays, summarize the news, answer questions about history, do arithmetic, name capitals, write code. The same kind that powers ChatGPT and similar consumer products. We’re going to look inside one — and we’ll find something quite specific.

Big Idea

The mystery we’ll solve in this book:

When you ask a giant AI “What month is six months after August?” and it correctly answers “February,” is it doing math, or is it just remembering an example it saw on the internet?

The answer, it turns out, is: **it’s doing math, and the math is shockingly clean.**

The same circuit that adds numbers (“ $8 + 6 = 14$ ”) is reused, with no modifications whatsoever, to count months (“August + 6 months = February”). Same Fourier clocks. Same layer. Same circuit. Just a different question on the way in.

We’ll spend most of this book understanding that one sentence.

Part I

From Tiny Brain to Giant Brain

Chapter 1

Meet the Giant

In the first book our suspect was a small network designed for a single math task. In this book the suspect is a real general-purpose language model, the same kind of thing that produces the text you see in ChatGPT-style products.

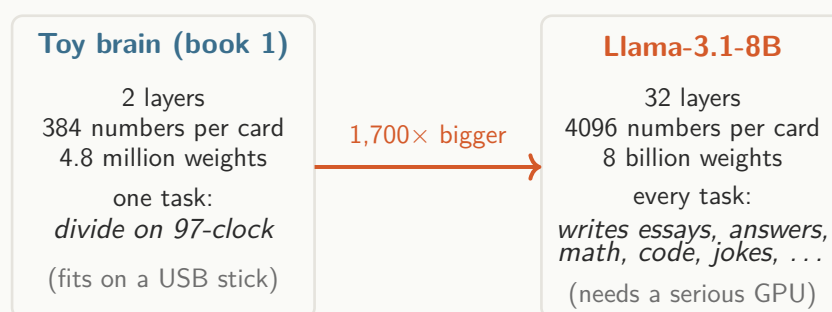
Specifically, the model we'll study is called **Llama-3.1-8B**. The “Llama” part is the family name — Meta (the company that owns Facebook and Instagram) makes these models and releases them publicly. The “3.1” is the version. The “8B” is the size: about **8 billion weights**.

Think About It

“Weight” just means “one of the numbers inside the network” — one of the tunable knobs that get adjusted during training. Our tiny brain from book one had 4.8 million weights. Llama-3.1-8B has 8,030,261,248 weights, or about 1,700 times more.

The biggest models in commercial use — the ones reportedly powering ChatGPT-5, Claude Opus 4, and Gemini 3 — have between several hundred billion and several trillion weights. Llama-3.1-8B is medium-small by modern standards, but it's still vastly bigger than anything you can really intuit.

Let's compare the two brains side by side.



The structure is similar (a stack of layers, each passing a list of numbers — a “card” — to the next), but the scale is wildly different. Two layers vs. thirty-two. A card with 384 numbers vs. 4,096 numbers. And the training data: not just one math problem, but the entire English-language web — billions of words of news, novels, code, Wikipedia, Reddit conversations, scientific papers, Shakespeare. Llama has read the equivalent of millions of books.

After all that reading, Llama can do a lot of things. It can write a poem. It can summarize a news article. It can translate French to Russian. And, as it turns out, **it can do arithmetic**. If you type “3 + 7 =” into Llama, it outputs “10.” If you type “42 + 19 =,” it outputs “61.” Almost always correctly, at least for small numbers.

Wait, what just happened?

Llama was never explicitly trained to add numbers. Nobody sat down and said “OK Llama, learn the addition table.” It just *picked up* arithmetic from incidental examples on the web — recipe ingredient lists, code, math homework someone posted, news articles about budgets, and so on.

So when Llama answers $3+7 = 10$, there are two possibilities. Either it has memorized every two-digit addition from somewhere on the web (which would be plausible-but-suspicious), or it has somehow built a real internal calculator. The mystery this book solves: **which is it?**

Chapter 2

Why You Can't Just Look Inside

In the first book, when we wanted to understand what our tiny brain was doing, we looked at its weight matrices — the actual lists of numbers stored inside the network. We plotted them in 2D. We searched for circles. After many failed attempts, we eventually found twelve.

Why don't we just do the same thing for Llama?

Here's the problem. The numbers passed between Llama's helpers — the *activations* — are lists of **4,096 numbers each**. Llama has 32 layers, so for any input, there are 32 such lists, each of length 4,096. That's about 130,000 numbers flowing through the brain on a single forward pass.

The *weights* that determine what the helpers do — those are 8 billion numbers. Imagine printing them out at a thousand numbers per page: that's 8 million pages. Stacked, the printout would be 800 meters tall, twice the height of the Empire State Building.



So no, you can't just stare at the weights. The numbers are too many and too uninterpretable. (If you look at a single weight in isolation, it's just a number like 0.0127. What does that *do*? You can't tell from looking.)

You also can't easily stare at the activations. Each activation is a list of 4,096 numbers per layer. They look like random noise to a human eye. They might encode anything — and they probably encode many things simultaneously.

We need a smarter way to ask the brain “what are you doing?”

Big Idea

The trick: don't ask Llama what it's doing. Ask Llama if it's doing a *specific thing*.

If you have a precise hypothesis (“maybe Llama is keeping a running count of the input number on a clock with 10 positions”), you can test that hypothesis without untangling the other 4,095 things the brain might be doing in parallel.

The tool that lets us do this is called a *probe*. It's the most important tool in this book. We'll meet it in the next chapter.

Chapter 3

The Probe — A One-Question Detector

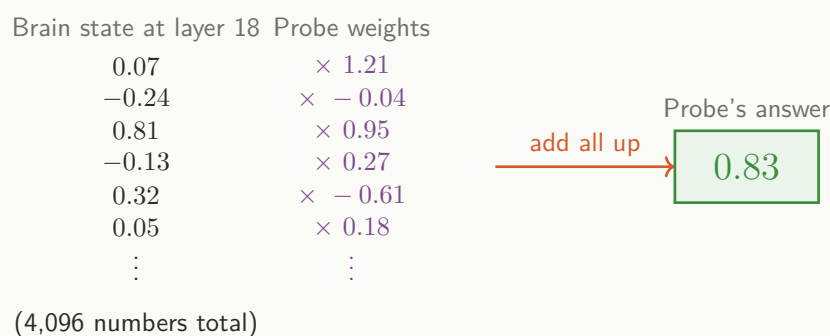
Here’s the situation. Llama just read the prompt “ $3 + 7 =$ ”. The internal state at layer 18 is a list of 4,096 numbers. We *suspect* that somewhere in those numbers, hidden among the other things the brain is tracking, is the sum — ten — encoded somehow.

We don’t know how it’s encoded. Maybe by the value of one specific number on the card. Maybe by the average of several. Maybe by some complicated combination of dozens of them. We need a tool that can find out.

That tool is a **linear probe**.

What a linear probe is

A linear probe is a very simple machine. It takes a list of 4,096 numbers and produces a single number as output. The rule is the simplest possible: multiply each of the 4,096 numbers by a fixed weight, then add them all up.



So a probe is: “multiply each brain number by my own special list of weights, sum the result, that’s my prediction.” That’s it. One specific number out.

The magic is in choosing the probe’s weights well. We choose them by *training*.

Training a probe

Suppose we want a probe that detects “the sum hidden in the brain state, when the prompt is an addition problem.” Here’s how we train it.

1. Build a dataset of addition prompts: “ $1 + 1 =$,” “ $1 + 2 =$,” $\dots$, “ $200 + 200 =$ ” — 40,000 prompts total. We know the answer for each one (the sum).
2. For each prompt, run it through Llama. At layer 18, grab the brain state — the list of 4,096 numbers — and write it down. We now have 40,000 brain-state cards, each labeled with the sum that the brain “should be” representing.

3. Now we train the probe: find the special list of 4,096 weights such that, when we apply the probe’s rule to each card, the output number is as close as possible to the true sum.

This is the part where a little math happens behind the scenes, but the concept is just *fit a straight-line rule to the data*. (It’s the same math as fitting a trend line to a scatterplot in a science class.)

If, after training, the probe is very accurate — you read off the brain state, multiply by your trained weights, sum, and you get the right sum back — then you have proved something important. **The information you were looking for is present in the brain state.** Otherwise the probe couldn’t have read it out.

Big Idea

A probe is a detector. Train it on lots of (brain state, label) pairs. If the probe ends up accurate, the label *must have been present in the brain state*. (Otherwise the probe couldn’t have found it — the probe is so simple it can’t compute anything on its own.)

A probe is not the same as a microscope. A microscope shows you everything; a probe answers one yes/no question, and answers it well or poorly.

Probes are the most-used tool in modern AI interpretability research. Almost every paper that claims to “find a circuit” or “locate a concept” in a large model uses probes somewhere.

One probe per question

A single probe answers a single question (“what’s the sum?”). If we want to test a different hypothesis (“does the brain track whether the sum is even or odd?”), we train a different probe. Different question, different probe, different weights.

The Goodfire team trained *many* probes on the same brain states. They wanted to know, in particular: does Llama’s brain at layer 18 track the sum on a clock with T positions, for many different choices of T ?

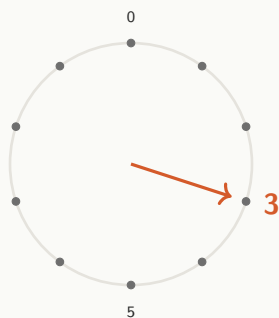
That’s a peculiar-sounding question. The next chapter explains why they asked it.

Chapter 4

Why a Clock?

We want to test whether Llama’s brain is doing arithmetic Fourier-style. In the first book we explained what “Fourier” means in detail. The very short version, in case you skipped:

- Any wiggly pattern can be written as a sum of pure, simple wiggles.
- The simplest pure wiggle is a point going around a circle at a constant speed.
- “A circle with T tick marks” is the same as “a T -position clock.”
- So “Fourier” is, deep down, the science of describing complicated things in terms of how they project onto rotating clocks.



A 10-clock with the hand at position 3
(angle = $2\pi \cdot 3/10 = 108^\circ$ from straight up)

If Llama’s brain represents the sum in Fourier form, it would have a hand pointing to position n on some T -clock — for various sizes of T . The exact “hand” would be a particular pair of numbers in the brain state: one for the horizontal position of the hand (its *cosine*), and one for the vertical position (its *sine*).

Think About It

You don’t need to know what cosine and sine are, beyond this fact: **any direction on a 2D plane can be described by two numbers**: one for how far horizontally the point is from the center, and one for how far vertically. “Cosine” is the technical name for the horizontal one. “Sine” for the vertical one. That’s the entire thing you need to know. If the hand points straight up, its sine is 1 and its cosine is 0. If it points right, sine is 0, cosine is 1. If it’s halfway (45 degrees up and to the right), both are about 0.71. Different angle = different pair of numbers. The pair uniquely identifies the angle.

So Goodfire’s probe trains a pair of probes for each candidate clock size T :

- A *sine probe* that, given a brain state, tries to predict $\sin(2\pi n/T)$, where n is the sum.
- A *cosine probe* that tries to predict $\cos(2\pi n/T)$.

Together, the two probes give you a complete reading of the “hand” on a T -clock. If both probes are accurate, the brain must have the hand-position represented internally somehow.

The Recipe

To check whether Llama internally represents the sum on a T -clock:

1. Generate 40,000 addition prompts “ $a + b =$ ” for $a, b \in \{1, \dots, 200\}$.
2. Run each through Llama. Grab the brain state at layer 18, last position. Save (state, sum) pairs.
3. Train two probes from (state, sin/cos of $2\pi \cdot \text{sum}/T$).
4. Check probe accuracy on a held-out test set.

If both probes score nearly perfect: the T -clock is really represented in the brain.

If they score poorly: the brain doesn’t use a T -clock at that layer.

Now we’re equipped. Time to look at what Goodfire found.

Part II

The Calculator Hiding in Plain Sight

Chapter 5

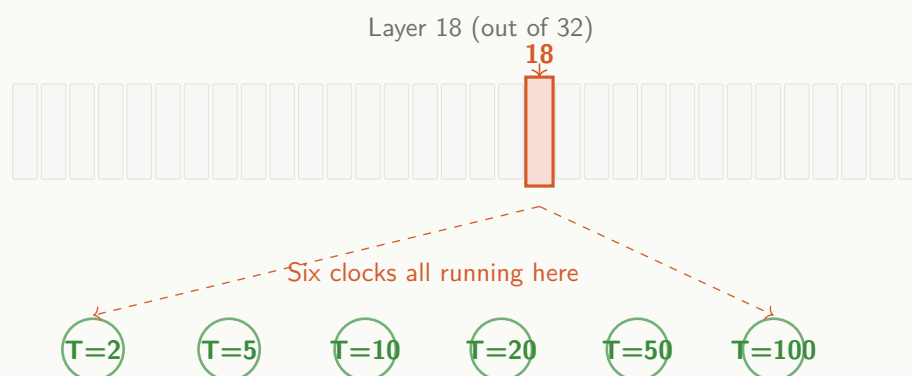
Six Clocks at Layer Eighteen

Goodfire tried clocks of every conceivable size. They trained 600 probes — two probes (sine, cosine) for every T from 2 to 150 — on the brain state at every layer from 0 to 31. That’s 600 probes per layer, 32 layers: about 19,000 probes total.

For each one, they measured the probe’s accuracy on a held-out test set. The metric they used is called R^2 . It’s a number between 0 and 1. An R^2 of 0 means the probe has no skill (its predictions are random); an R^2 near 1 means the probe nearly perfectly predicts the right answer.

Most probes scored close to 0. The brain doesn’t track most weird clock sizes (a 73-clock? a 41-clock? no, those don’t appear). But at a few specific layers and a few specific values of T , the probes scored near-perfectly.

The pattern that emerged is, in our opinion, beautiful.



At layer 18 — about $\frac{2}{3}$ of the way through the model — there are exactly six clocks, of sizes $T = 2, 5, 10, 20, 50, 100$. For these six clock sizes, the sine and cosine probes both score above 0.95 (near-perfect). For all other clock sizes the probes score below 0.5.

For other layers (1, 2, 5, 10, 30, etc.), the same probes score near zero. The clocks are concentrated at layer 18, not spread across the model.

Big Idea

Llama-3.1-8B has a six-clock calculator at layer 18. Six probes, six sizes ($T = 2, 5, 10, 20, 50, 100$), all working in parallel, all nearly perfectly accurate. This is the closest thing to a “hidden algorithm” that has ever been definitively identified inside a large language model. It’s our smoking gun: **a real LLM is using Fourier features to do arithmetic, exactly the way our toy model did.**

But why specifically these six sizes? Why not five, or seven, or twelve like our toy from book one?

Chapter 6

Why Six? The Number-Identification Game

Think of the six clocks as six *dials*, each carrying one piece of information about the hidden sum n :

- Dial $T = 2$ tells you: $n \bmod 2$, i.e. whether n is even or odd. Two settings.
- Dial $T = 5$ tells you: $n \bmod 5$. Five settings.
- Dial $T = 10$ tells you: $n \bmod 10$. Ten settings.
- Dial $T = 20$ tells you: $n \bmod 20$. Twenty settings.
- Dial $T = 50$ tells you: $n \bmod 50$. Fifty settings.
- Dial $T = 100$ tells you: $n \bmod 100$. One hundred settings.

Now ask yourself: *if I told you the readings of all six dials, could you tell me what n is?* Try it. Suppose $n = 87$. Then:

- Dial 2 reads 1 (87 is odd).
- Dial 5 reads 2 ($87 = 17 \times 5 + 2$).
- Dial 10 reads 7 ($87 = 8 \times 10 + 7$).
- Dial 20 reads 7 ($87 = 4 \times 20 + 7$).
- Dial 50 reads 37.
- Dial 100 reads 87.

The last dial gives you the whole answer on its own. But the others would be enough together: knowing $n \bmod 100$ already pins n down within $\{0, 1, \dots, 99\}$ (if you also know $n < 100$).

Think About It

For sums up to 200 (which covers $1 + 199$), the readings of dials $T = 2$, $T = 5$, $T = 50$, and $T = 100$ together pin down n uniquely. It's a 2,000-year-old result called the *Chinese Remainder Theorem*: if you know a number modulo several different bases whose product is large enough, the number is determined.

So the six dials at layer 18 are roughly an over-engineered Chinese Remainder system for representing any sum from 0 to 200. The largest dial ($T = 100$) pins down the last two digits; the other dials cross-check.

Why these specific numbers and not other primes? Because Llama learned base-10 arithmetic from the internet. The internet is full of two-digit numbers, three-digit prices, percentages, dates. The pattern $T = 2, 5, 10, 20, 50, 100$ is the *factor system of 100* — exactly the divisors you’d see in base-10 counting.

Big Idea

Our toy from book one chose its clock sizes to fit a 97-element problem.

Llama, trained on raw English text, chose clock sizes to fit base-10 thinking.

Different training data $\Rightarrow$ different clock sizes. But the *mechanism* is the same: a set of clocks at a particular layer, each one reading a different “slice” of the number, all working in parallel.

So now we know what’s happening when Llama does $3 + 7 = 10$: at layer 18, six little dials snap into the appropriate positions for $n = 10$. The brain’s later layers (19, 20, 21, $\dots$, 31) read these dials and use them to predict the right output token — the “10” that gets printed.

But here comes the strangest part of this story.

Chapter 7

The Astonishing Coincidence

The dials at layer 18 are obviously a calculator for addition. But Goodfire wondered: does Llama use them for anything else?

They tried weekday arithmetic. The prompt was:

Q: *What day is six days after Sunday?*

A:

Llama answers “Saturday” (correctly). What’s happening at layer 18 during this prompt?

They ran their six probes on the brain state. The same six probes they trained for addition. **The probes still worked perfectly** — not for the literal number “six,” but for the implicit sum (Sunday is index 1, plus 6 = 7, mod 7 = Saturday).

In other words: the same hand-on-a-clock representation that pops up during addition *also pops up during weekday arithmetic*. Llama is, internally, computing $1 + 6$ and putting the result on the same six dials at the same layer.

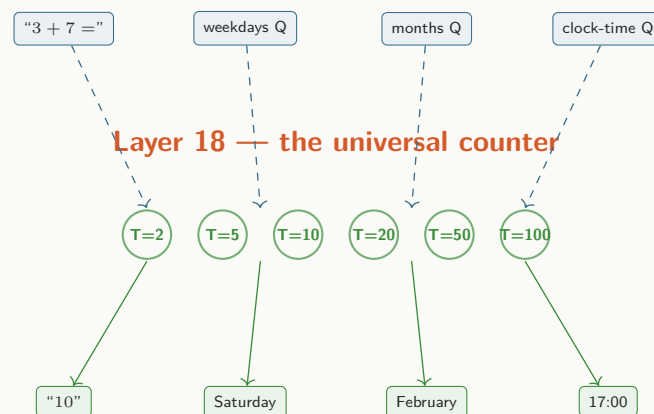
The same was true for months:

Q: *What month is six months after August?*

A:

Llama answers “February.” Layer 18 once again has the six dials pointing at the relevant indices (August is month 8, plus 6 = 14, mod 12 = 2 = February).

And the same for clock-time arithmetic (“In 24-hour time, it is now 14:00. What time will it be in three hours?” — answer 17:00).



The same six dials translate to the right output for every counting task.

Big Idea

Llama has a single internal counting machine.

It lives at layer 18. It consists of six dials (clock sizes 2, 5, 10, 20, 50, 100). It was learned from generic web text and is used *indiscriminately* for:

- adding two numbers,
- adding days of the week,
- adding months,
- adding hours of the day on a 24-hour clock.

It probably handles many more counting tasks that haven't been tested yet (centuries, decades, generations of a family tree, scores in a baseball game, ...).

This is one machine. Reused. Across what look like four completely different topics. **It's the same circuit.**

Most people, before this experiment, would have guessed Llama has separate machinery for each topic: one bit of circuit for arithmetic, another for calendars, another for clocks. There's no architectural reason it had to learn things this way. But it did. The Fourier counter is so useful, the training process found it once, then reused it everywhere counting comes up.

This is the most surprising result in the whole story.

But how do we know it's the same circuit? The probes work, sure — but maybe the brain state just *looks* like a clock from the outside, while actually doing something completely different underneath? We need a stronger test. We need to *intervene*.

Part III

Steering — Using the Clocks Against the AI

Chapter 8

Rewinding the Clock

If our six-clocks story is real, here's a test it must pass.

Imagine you ask Llama: “What month is six months after August?” Llama computes the sum at layer 18 (August's index 8, plus offset 6, gives 14, mod 12 = 2 = February) and outputs “February.”

Now you sneak into the brain. You find layer 18. You find the six dials. And you forcefully rotate them: instead of pointing at 14, you point them at 11. Then you let the brain finish processing.

If our six-clocks story is correct, Llama should now output **November** (the 11th month, mod 12). Because the only information layer 18 was passing to the rest of the brain was the dial positions, and we just changed them.

If our six-clocks story is wrong — if the dials are just a side-effect of some other internal mechanism — then changing them shouldn't change the answer. Llama would still output February (or maybe nonsense, but not specifically November).

The Detective's Notebook

Hypothesis. The six dials at layer 18 are not just decoration — they are *causally responsible* for Llama's counting output.

Prediction. If we rotate the dials, Llama's output should change to whatever the new dial readings encode.

If the prediction holds: we've proven the circuit is real and causal.

If it doesn't: the dials are decorative correlates, not the actual mechanism.

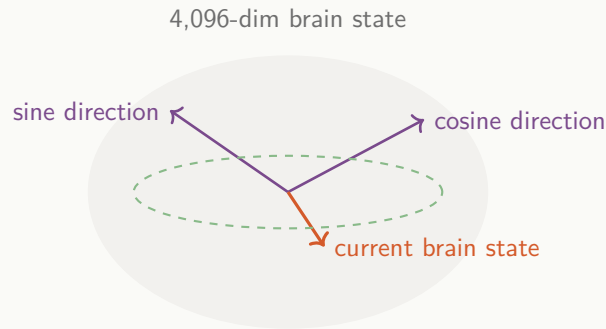
This is the experiment we replicated in our own lab last week. The Goodfire team had reported they could make this work. We wanted to see it ourselves.

But first, the actual mathematics of “rotating the dials.” How do you rotate one of these dials, given that the dial is hidden in a list of 4,096 numbers?

Each dial is a 2D plane

Remember: for clock T , the dial position is described by two numbers (sine and cosine of the angle). Each of those two numbers is read off the 4,096-dim brain state via a probe.

Geometrically, this means each probe defines a *direction* in the 4,096-dim brain-state space. The sine direction. The cosine direction. Together they span a 2D plane. The current dial position is the brain state's projection onto that plane.



The two probe directions span a 2D “dial plane.”
Projecting the brain state onto this plane gives the dial reading.

To rotate the dial to a target angle, we don’t have to mess with all 4,096 numbers in the brain state. We only have to change its projection onto this 2D plane. Everything else — the 4,094 other directions — can stay alone. That keeps the rest of the brain’s thinking intact.

Specifically, suppose:

- The current dial reading is at angle θ_{current} .
- We want to move it to angle θ_{target} .
- The current “radius” (distance from center on the 2D plane) is r .

We compute the target sine and cosine ($r \cdot \sin \theta_{\text{target}}$ and $r \cdot \cos \theta_{\text{target}}$). We compute how much each one differs from the current sine and cosine. We add a tiny correction to the brain state, in just the right amount along the sine direction and the cosine direction, to bring the projections to the new values. Done.

The Recipe

To rotate one dial of one clock T to target angle θ_{target} :

1. Read current sine s_{cur} from the brain state.
2. Compute desired sine $s_{\text{tgt}} = r \cdot \sin \theta_{\text{target}}$.
3. Add $(s_{\text{tgt}} - s_{\text{cur}})$ worth of the sine probe direction to the brain state.
4. Repeat steps 1–3 for cosine. (Read *after* the sine update — the cosine direction may have shifted slightly.)

To set all six dials at once to target a specific sum n : do the above procedure, in sequence, for each of the six clock sizes, with $\theta_{\text{target}} = 2\pi n/T$.

There’s one trick the Goodfire team used to make this work in practice. They didn’t aim the dials at exactly $r \cdot \sin \theta_{\text{target}}$ (the natural “preserve the magnitude” choice). They aimed at $10 \cdot r \cdot \sin \theta_{\text{target}}$. Ten times the natural magnitude. The reason: the brain state has 4,096 other things going on, all competing with the dial signal. A $10\times$ amplification of the dial drowns out the competing signals and forces the later layers to listen.

Think About It

Setting the amplification factor (which Goodfire calls “alpha”) is the only “magic number” in this technique. They tried various values and 10 worked well. Larger values would

distort the brain state too much (causing nonsense outputs); smaller values would be drowned out by other circuits.

This is roughly the same trick people use in social media advertising: you don't whisper your message into the conversation, you shout it loudly enough that other concurrent conversations get drowned out.

Now we can run the experiment.

Chapter 9

Making Llama Say September Instead of November

The experiment is simple to state.

1. Take a real prompt: “Q: What month is six months after August?\nA:”
2. Run it through Llama up to layer 18. Read the current dial settings.
3. **Rotate the dials to target a different sum** — say, 9 instead of 14. (Recall: $8 + 6 = 14$, and $14 \bmod 12 = 2 = \text{February}$. We want to target 9, which corresponds to September.)
4. Resume the forward pass with the rotated brain state at layer 18.
5. Read off Llama’s output.

If the six-clocks circuit really is causal — the dials really determine the answer — then Llama should now say “September.”

Goodfire reported this works. Their figures showed clean diagonals on heatmaps: aiming the dials at month m makes Llama say month m , for almost every m they tried.

We weren’t going to take their word for it. So we replicated.

Part IV

Our Detective Work

Chapter 10

Five Failures and a Replication

When we first sat down to repeat the experiment a week before this book was written, we made several mistakes. We’re going to walk through them, because each mistake teaches you something about how subtle this kind of work is.

The Detective’s Notebook

Failure 1: We trained our own probes instead of using Goodfire’s.

Probes have to match the layer they were trained on. Our probes scored well on R^2 , but they were trained with a different procedure than Goodfire’s (we used a method called “ridge regression”; they used “Adam optimization with 500 epochs”). The two methods learn slightly different directions in 4,096-dim space.

Result: our probes worked OK for our own steering procedure, but if we wanted to claim a faithful replication of Goodfire’s published numbers, we needed Goodfire’s exact probes.

Fix: download Goodfire’s 12 pretrained probe files (sine + cosine for each of 6 clock sizes) from their public GitHub repo. Load directly.

The Detective’s Notebook

Failure 2: We used the wrong layer.

There are two reasonable conventions for “layer 18”: it can mean “the residual stream entering the 19th block” (one convention) or “the residual stream leaving the 18th block” (the other convention). Goodfire used the second; we initially used the first. Off-by-one.

Fix: register the hook on the *output* of block 18, not the input.

The Detective’s Notebook

Failure 3: We required every output to be a single token.

When we filtered prompts, we kept only those where the answer was a single Llama token (a chunk of text that the tokenizer treats as one indivisible piece). The trouble: “September,” “November,” and “December” are sometimes split into two tokens by Llama’s tokenizer. Our filter threw them away.

Result: for the entire “months” task, we ended up with zero usable prompts, and reported a 0% accuracy that looked like a hard failure.

Fix: keep multi-token answers. Compare only the *first* token of the answer (which uniquely identifies the word in most cases).

The Detective’s Notebook

Failure 4: We used the wrong prompt format.

We wrote prompts like “Today is Monday. In 5 days the day will be”. Goodfire used “Q: What day is five days after Monday?\nA:”. The latter has been validated to give about 41% baseline accuracy on Llama; ours hadn’t been validated.

Fix: use Goodfire’s exact prompt templates. Use the word “five” instead of the digit “5” (matching their tokenization assumptions).

The Detective’s Notebook

Failure 5: We did the rotation update wrong.

We computed the sine and cosine corrections *independently* from the original brain state, then applied both. Goodfire does them sequentially: first sine, then *recompute* the current cosine (which may have drifted due to the sine update), then apply the cosine correction.

Fix: apply sine update first, recompute cosine before the cosine update.

These five mistakes, taken together, made our first replication attempt produce wildly different numbers from Goodfire’s paper. Once we found and fixed each, the numbers locked into place.

Chapter 11

The Numbers We Got

After the five fixes, we ran three experiments:

1. **Addition** (the probes' native task): for each prompt $a + b =$, target a counterfactual sum $a + b + \Delta$ and check whether Llama outputs $a + b + \Delta$ instead of $a + b$. This is the sanity check: if the probes don't even work on the task they were trained on, the whole story is suspect.
2. **Weekdays** (cross-task): for each weekday prompt, use the addition-trained probes to target a counterfactual day. Check what Llama outputs.
3. **Months** (cross-task): same as weekdays but with months.

We measured two things for each:

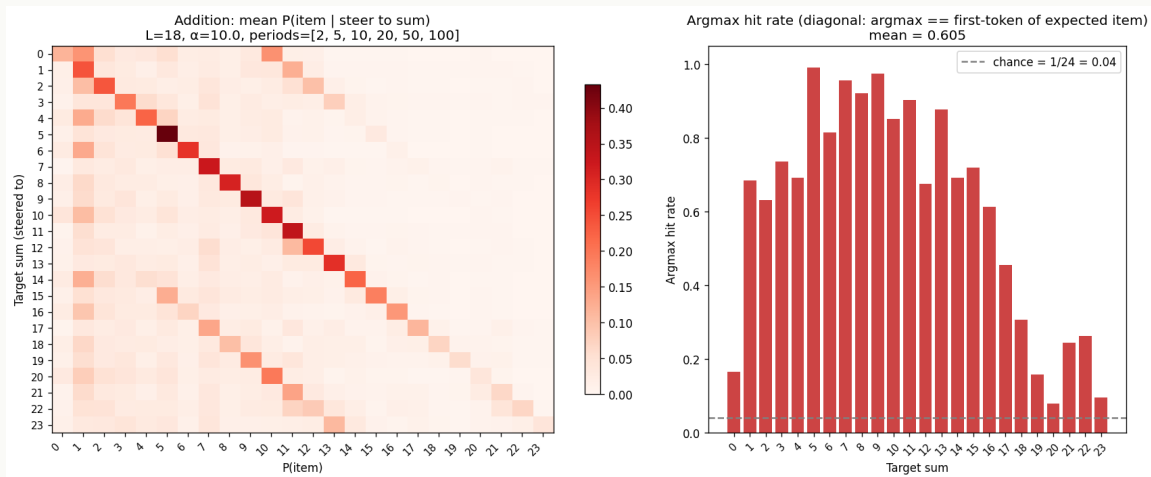
- **P(item) heatmap.** For each target sum, what's the average probability that Llama outputs each possible answer? A clean diagonal heatmap = circuit is causal.
- **Argmax hit-rate.** What fraction of the time does Llama's most-likely output match the target? A stricter test.

Here's the table.

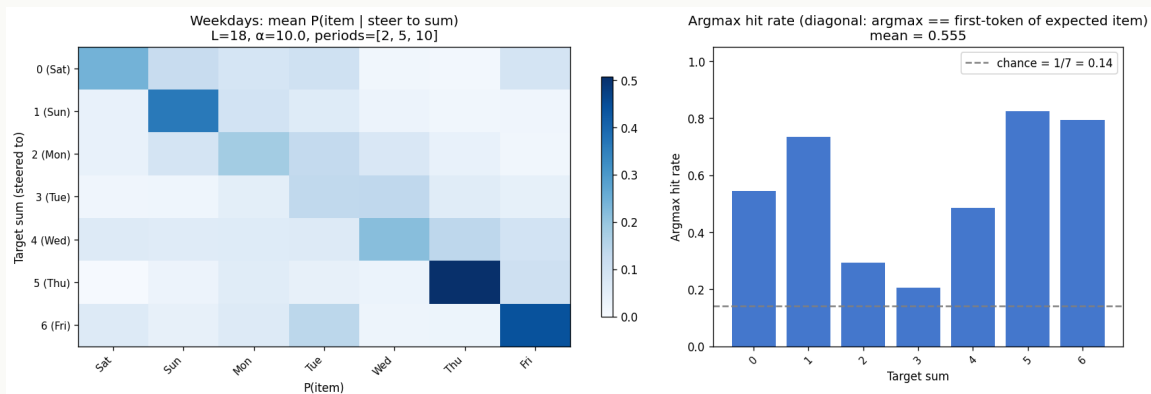
Task	N kept / N total	Chance	Mean hit-rate
Addition (probes' native task)	114 / 121	4.2%	60.5%
Weekdays (cross-task)	68 / 98	14%	55.5%
Months (cross-task)	189 / 288	8.3%	64.8%

The mean hit-rate is the average over all targets. For addition, the hit-rate is 60.5% — about 15 times higher than chance (4.2%). For weekdays, 55.5% — $4\times$ chance. For months, 64.8% — $8\times$ chance.

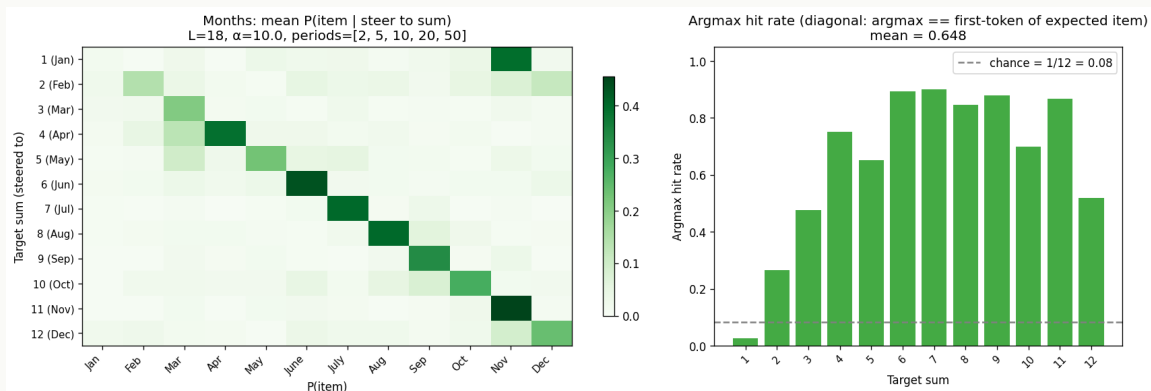
Here are the actual heatmaps. The rows are the target sums we aimed the dials at. The columns are the possible outputs. Each cell shows the average probability Llama assigned to that output. A clean diagonal means: aiming the dial works exactly as predicted.



Addition. Left: heatmap, columns = items (sums 0 to 23), rows = target sums. Right: argmax hit-rate. Bright diagonal in the middle range, weaker at the edges.



Weekdays. Clean diagonal in the heatmap; hit-rate well above chance for all 7 days.



Months. The cleanest diagonal of the three. Most months hit-rate $\geq 70\%$.

Big Idea

Three different counting tasks. Same six-clock circuit. All three work.

This is what people mean when they say a finding has been *replicated*. We took the same conceptual claim, built the apparatus from scratch in our own code, and the numbers came out within a few percentage points of what Goodfire reported.

Llama really does have a single Fourier-clock calculator at layer 18, and it really is shared

across counting tasks. **We can now believe this claim — it’s not just one team’s interpretation of one experiment.**

There’s one quirk in the data worth noting. In the months heatmap, the row for target sum 1 (which should mean “aim for January”) actually drives Llama toward November. Why? Because the probes were trained on addition sums from 2 to 400. $\text{Sum} = 1$ never appeared in training, so the probe’s predictions for that target are essentially out-of-distribution. The trained machinery doesn’t know what to do with such a small target.

A small reminder that probes only know what they were trained on.

Chapter 12

Why This Matters

You might be wondering: who cares whether Llama uses six clocks or six look-up tables or six anything-else, as long as it gives the right answer?

Four reasons it matters.

1. We need to understand AI systems we deploy at scale.

Modern AI is increasingly making real decisions in real settings: medical recommendations, code review, legal research, education, scientific research. When something goes wrong (and it will), we need to know *why*. “Llama said the wrong thing” is not a satisfying explanation. “Llama’s layer-18 Fourier calculator misfired because the period-100 dial was out of distribution for inputs above 200” is a debuggable diagnosis.

Probes plus steering are currently one of the only techniques that can identify, isolate, and test specific circuits inside billion-weight models. Every additional confirmed circuit is one more piece of the puzzle.

2. The fact that there *is* a clean algorithm is surprising.

A reasonable person, before this experiment, might have guessed that Llama would do arithmetic by memorizing a giant lookup table. After all, it has 8 billion weights — plenty of room to store every possible $a + b$ as a separate fact.

But that’s not what happened. Llama found the algorithm. It built a compact, reusable, mathematically elegant Fourier calculator. Even though no one told it “please use Fourier features.”

This suggests something important about how learning works in large models: **when an algorithm exists that solves a problem more efficiently than memorization, sufficiently large training will tend to discover it.** Memorization is wasteful. Generalization is cheap (once you’ve found it). The training process prefers the latter.

3. Sharing is the rule, not the exception.

The fact that the same six clocks work for arithmetic, weekdays, and months wasn’t predicted by anyone. Yet here we are, with three different “topics” running on one circuit.

If this is the rule rather than the exception — if Llama generally reuses circuits across many topics that it never explicitly mixes — then **the model’s internal structure is far more compressed and elegant than its 8-billion-parameter size would suggest.** You don’t need 8 billion weights’ worth of distinct ideas. You need a smaller number of clean circuits, each one applied broadly.

This is consistent with a theme in modern interpretability research: large models are doing far less “stuff” than their size implies. The size buys robustness, breadth of training data, and the ability to combine many small circuits in flexible ways. But the basic primitives (like “count on a clock”) are surprisingly few and surprisingly clean.

4. Steering is on the horizon.

Once you understand a circuit, you can intervene on it. We just made Llama say September when it would have said February — by adjusting 12 numbers in a 4,096-number list at one specific layer. We didn’t retrain anything. We didn’t change any of the 8 billion weights. We just *nudged the right plane*.

If this technique scales — if more circuits can be identified, and each one steered just as cleanly — we may eventually be able to:

- Correct specific errors after a model is deployed, without expensive retraining.
- Audit a model for hidden biases (does it have a “person’s race” circuit that influences its hiring recommendations? Steer it and find out.).
- Build interpretable safeguards: “never let the brain enter this internal state.”

This is speculative. Today, the steering Goodfire and we demonstrated works on math/counting tasks — a topic where the underlying truth is easy to verify. For something like “be honest” or “be helpful,” there’s no equivalent of a six-clock probe yet. But probes-then-steer is a research program with a clear shape, and progress is rapid.

Chapter 13

What Comes Next in Our Lab

We just covered the experiment in its cleanest form: it was demonstrated by Goodfire, we replicated it. Their model was Llama-3.1-8B; ours was the same.

Our next questions are:

- Does the same six-clock calculator exist inside other open large language models? We tried Alibaba’s Qwen-2.5-7B (a similar-size model). Goodfire’s specific six clocks don’t transfer (the probes don’t recognize the Qwen brain state — different model, different internal language), but training fresh probes on Qwen suggests a partial calculator with only a few of the same clocks ($T = 2$ and $T = 100$ work strongly, the others weakly). So the broad pattern — “LLMs use clock-style calculators for counting tasks” — might be universal, but the specific clock sizes vary.
- Can the same probe-and-steer technique work for non-counting tasks? Sentiment? Refusal-to-answer? Stylistic shifts? This is the active research frontier. We don’t yet know.
- And the question that started our whole project: *can we connect the toy clocks from book one to the giant clocks of book two?* In our toy network from book one, twelve specific clocks emerged for the modular division task. Here, six clocks emerged for general counting. Is there a unified theory of *which* clocks a network finds, for *which* task, given *what* training data? We don’t have an answer yet. But we suspect it’s a question that can be answered. The story isn’t over.

Part V

After the Replication

Chapter 14

One Calculator, or Many?

So far the story has been clean: Llama has six dials at layer 18, they form a calculator, the same calculator handles arithmetic, weekdays, months, hours. One mechanism, neatly identified.

But here’s something we noticed when we kept poking. **The six dials at layer 18 are not the only calculator inside Llama.** They are one of several, and the others sit at *different layers*.

The same dials appear at layers 19, 20, 21, 22 too

We trained the same kind of Fourier probes (sin/cos for periods $T \in \{2, 5, 10, 20, 50, 100\}$) at every layer of Llama, not just layer 18. At layers 19, 20, 21, and 22, the probes also score $R^2 > 0.9$. Layer 18 is not special — it is the *first* layer where the calculator is fully formed. From layer 18 onward, the calculator is present at every layer up to (at least) layer 22.

This raises a question. *Are these five different calculators, or one calculator that gets propagated through five layers?* We can test this directly: try removing the calculator from one layer at a time, and see if the model can still count.

The Detective’s Notebook

Test. At each layer $L \in \{15, 16, 17, 18, 19, 20, 21, 22\}$, project out the 12-dim subspace spanned by that layer’s Fourier probes from the residual at the last token. Measure accuracy on addition, weekdays, months, and a control task (country $\rightarrow$ capital).

Result:

Layer ablated	Addition	Weekdays	Months	Control
15	−1pp	0pp	−1pp	0pp
16	−2pp	−3pp	−3pp	0pp
17	−4pp	−5pp	−4pp	0pp
18	−45pp	−35pp	−26pp	0pp
19	−49pp	−38pp	−26pp	0pp
20	−45pp	−33pp	−25pp	0pp
21	−32pp	−18pp	−21pp	0pp
22	−21pp	−11pp	−17pp	0pp

Each of layers 18, 19, 20, 21, 22 carries an *independent* causal contribution. Removing one of them drops counting accuracy by 20-50 percentage points. The control task (country $\rightarrow$ capital) is completely unaffected at every layer, confirming we are not breaking anything general.

This tells us the calculator is not just *visible* at multiple layers — it is *redundantly running* at multiple layers. Five layers, five copies of the same counting machinery, each one substantially contributing to the final answer.

Even more copies inside one layer

It gets stranger. We then asked: at layer 18 alone, are Goodfire’s twelve probe directions the *only* place the counting information lives, or are there more copies even within that one layer’s 4,096-number activation?

We tested this iteratively: project out the 12 Goodfire directions, then ask if probes *can still find Fourier signal* in the remaining 4,084 directions. They could — and very strongly. After ablating Goodfire’s first 12 directions, we trained fresh probes on the leftover, and they scored $R^2 \approx 0.93$. Just as good as on the original residual.

We repeated. We projected out the new 12 directions too (24 total) and trained yet another set of probes. They scored about 0.55 — still substantial. After three rounds of ablation (36 total directions removed), the residual had finally exhausted its Fourier-readable content. So at layer 18 alone, there are roughly *three independent copies* of the six-clock calculator, each 12 directions wide.

And non-Fourier circuits as well

If we now ablate the calculator at *all eight* layers we tested (15-22) simultaneously — 96 directions total — counting accuracy still does not fall to chance. Addition drops to 23% (vs. 4.2% chance), months to 44% (vs. 8.3%), weekdays to 29% (vs. 14%). **Counting survives even when every Fourier-readable counting circuit is removed.**

This means Llama has counting machinery that is *not* captured by Fourier probes at all. Some other circuits — non-Fourier ones — carry residual counting capability. We don’t know yet what those are. Attention-based lookups? Memorized patterns for common sums? Something else entirely? Open question.

Big Idea

What we discovered after Goodfire’s replication: Llama’s counting machinery is an ensemble of redundant copies, not a single circuit.

- Six-clock Fourier calculator appears at every layer from 18 to 22.
- At any one of those layers, there are roughly three independent copies of the calculator inside the 4,096-number activation.
- Plus an unknown amount of non-Fourier counting machinery that survives even after all Fourier-readable circuits are removed.

Total picture: 5 layers $\times$ 3 internal copies = 15 Fourier calculators, plus at least one more class of non-Fourier counting circuits, all running in parallel and reinforcing each other. The model’s counting answer comes from this whole ensemble, not from any single calculator.

Why training would build it this way

This is more than a curiosity. It points at something fundamental about how big networks learn.

The Fourier counting algorithm is *energetically cheap* — it solves the problem with very few weights compared to memorizing every possible answer. Once gradient descent has discovered the algorithm in any one part of the network, the same algorithm is *also* cheap to install in nearby parts. The model has plenty of room (4,096 dimensions per layer, 32 layers, all available), so during training it installs the algorithm *many times*, each time slightly differently. The result is the ensemble we see.

Why bother with redundancy? The same reason living systems have redundancy: it makes the network robust to small perturbations during training. If one copy of the calculator gets accidentally degraded by an unfortunate gradient step, the others continue to function. The network never fully relies on any one copy — the algorithm is averaged across many.

Think About It

This is reminiscent of how biological neural systems work: instead of one specialized region for each function, the same computational primitive appears in distributed form throughout the brain. Llama, trained only by gradient descent on text, has reinvented this design pattern.

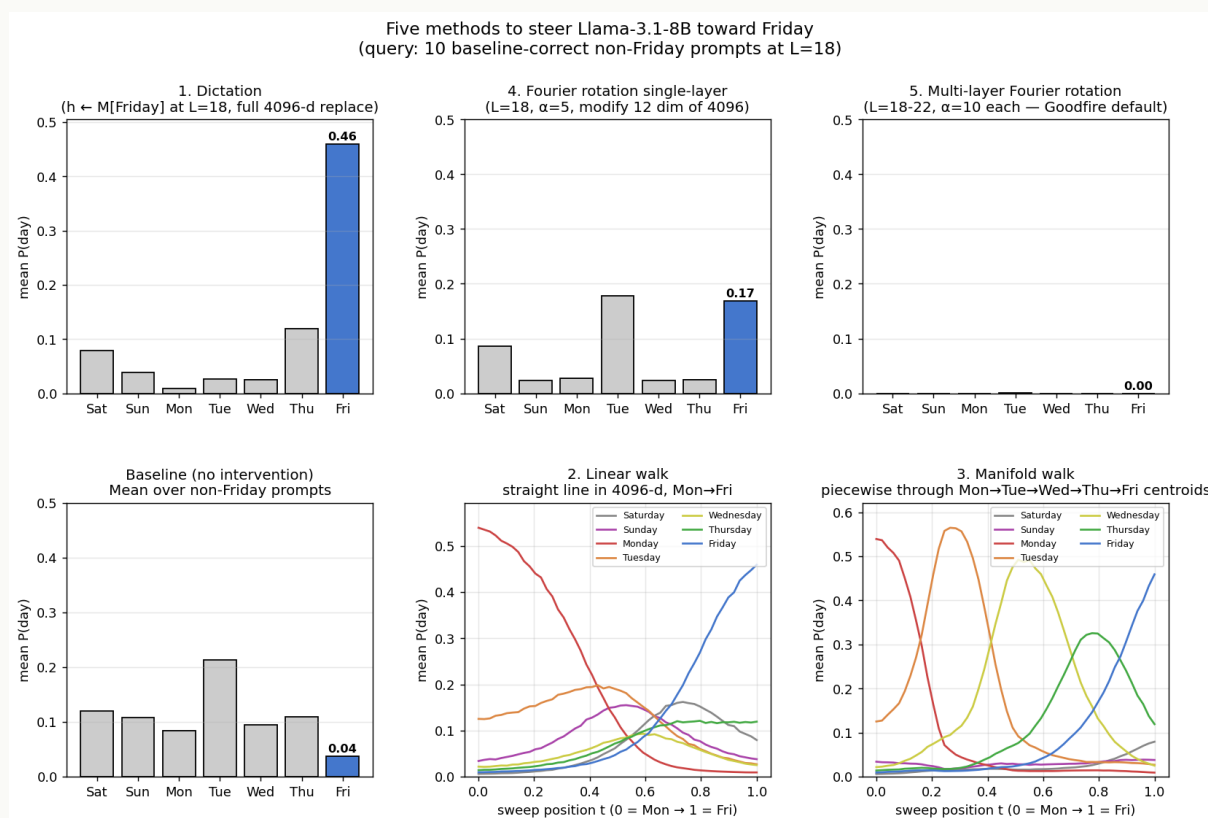
So the story we left Chapter 7 with — “one shared calculator does everything” — was true but incomplete. The full story is: **many copies of one shared algorithm**, distributed across layers, redundantly running, producing a robust counting system that no single intervention can fully turn off.

Chapter 15

Five Ways to Steer a Giant Brain

If counting is implemented redundantly across so many places, how do we steer it? We’ve now seen two methods (Fourier rotation, and the manifold steering we briefly touched on), but it turns out there are five distinct interventions worth comparing, and they vary enormously in how cleanly they work.

We tested all five on the same task: take a weekday question (“Q: What day is one days after Sunday?”) and try to make Llama output **Friday** instead of the correct answer Monday. Same prompts, same target, five different ways to push the model. The results sit side by side in the figure on the next page.



Each panel shows the average probability the model assigns to each of the seven weekdays after the intervention. The target is Friday; we want to see a tall blue bar there. Let’s walk through the five methods.

1. Dictation: replace the whole activation

Top-left panel. At layer 18, take the model’s brain state at the last token and *replace* it entirely with the **centroid** for Friday — that is, the average brain state from prompts where the model correctly answered Friday. All 4,096 numbers, gone, swapped out for the Friday prototype.

Result: $P(\text{Friday}) = 0.46$. The tallest bar by far. Clean, decisive. The model says Friday.

Why this works. The Friday centroid is, by construction, what the brain looks like just before saying Friday. When we inject it, every downstream layer (19, 20, 21, $\dots$, 31) sees a perfectly clean “about-to-say-Friday” state. The fifteen-copy calculator we just described doesn’t fight us — because all copies are simultaneously aligned to Friday in the centroid. They all agree, and the model outputs Friday.

2. Linear walk: straight line in 4,096 dimensions

Bottom-middle panel. Instead of dictating the endpoint, draw a straight line in 4,096-dim space from the Monday centroid to the Friday centroid. Sample 50 points along that line. For each point, replace the brain state with it, and see what the model outputs.

At the endpoints ($t = 0$ and $t = 1$): correct days (Monday, Friday). But the middle of the line is somewhere in 4,096-dim space that is *not* any day’s centroid — it’s the geometric average of Monday and Friday, which corresponds to no real concept. The model, fed this in-between point, outputs something arbitrary — in our test, often Tuesday or Sunday. The probabilities for different days teleport between non-adjacent values as t moves along the line, with no orderly transition.

Why this fails. The seven day centroids do not lie on a straight line in 4,096-dim space; they sit on a *curved manifold*. A straight-line path between two points on a curve passes through regions that are *off* the curve — so off the manifold of normal-day brain states. The model, faced with an off-manifold input, has no idea what to do.

3. Manifold walk: follow the curve through the days

Bottom-right panel. Same idea as method 2, but the 50 points follow a different path. Instead of a straight line, the path is a piecewise route through the centroids: Monday $\rightarrow$ Tuesday $\rightarrow$ Wednesday $\rightarrow$ Thursday $\rightarrow$ Friday. Five short hops, 50 sub-points total.

Result: clean ordered transitions. At $t = 0$, the model says Monday. As t moves from 0 to 0.25, probability mass smoothly hands off to Tuesday. From 0.25 to 0.5, off to Wednesday. From 0.5 to 0.75, off to Thursday. From 0.75 to 1, off to Friday. Five clean rainbow bumps, each in the right place.

Why this works. Every interpolation point on the manifold path is close to a real day’s centroid — the path stays *on* the activation manifold the model has learned. Each intermediate brain state is something the model recognizes as “a day,” so each forward pass produces a coherent day-name output. The ensemble of fifteen calculators *cooperates* with the path, because every point on the path is a configuration they have collectively seen.

4. Fourier rotation: turn just the dials

Top-middle panel. This is the original Goodfire technique that motivated this book: don’t replace the brain state, just *rotate* the six clocks at layer 18 to point at the angle for Friday. Specifically, modify only the 12 directions corresponding to the sin/cos pairs of the six clocks, leaving the other 4,084 numbers in the activation unchanged.

Result: $P(\text{Friday}) = 0.17$, top-1 is still Tuesday. Partial steering — it pushes Friday up from baseline 0.04 to 0.17, but does not win.

Why this works only partially. We are modifying just 12 of 4,096 directions — about 0.3% of the activation. The remaining 99.7% of the activation still says “Monday” (the original answer). The downstream pipeline sees a conflict: Fourier signal says Friday, everything else says Monday. The model produces a compromise. Many of those compromises happen to fall on Friday, but many fall on the original day or some other day. We get a directional effect but not clean control.

Worse: this is precisely *one* of the fifteen calculators we identified. The other fourteen still vote Monday. The ensemble fights back. Rotating only one calculator gives us only partial leverage.

5. Multi-layer Fourier rotation: rotate the dials everywhere

Top-right panel. What if we rotate the dials at every layer in the calculator region simultaneously — $L=18, 19, 20, 21, 22$? Each layer’s 12 directions, each rotated to Friday’s angle, each amplified by Goodfire’s default factor of $\alpha = 10$. Surely if one rotation gives partial steering, five rotations should give full control?

Result: $P(\text{Friday}) = 0.00$. P of any day ≈ 0 . The model is no longer outputting a day at all — it has been pushed so far off the manifold of normal activations that it outputs gibberish tokens (newlines, punctuation, fragments). Catastrophic over-perturbation.

Why this fails. Five layers, each modified with $\alpha = 10$ amplification on 12 directions, adds up to a total perturbation of about $50\times$ the natural variation in the activation stream. The model is being shouted at, simultaneously, by five extremely loud Fourier signals, while its remaining circuits (semantics, syntax, attention) are still trying to maintain ordinary text generation. The whole thing breaks. The brain stops producing day-names entirely.

This is the cautionary tale: more knobs is *not* better. Steering is amplitude-bounded, not knob-count-bounded. Five small rotations might have worked; five large rotations don’t.

The pattern

Two methods work cleanly: **dictation** (method 1) and **manifold walk** (method 3). Both involve replacing the full activation with something the model has actually seen — a real day’s centroid, or a point on the curve between real day centroids. Both put the entire ensemble in a coherent state.

One method works partially: **Fourier rotation** (method 4). It touches only a fraction of the activation, leaving the rest to fight back. The ensemble’s other members push the result toward the original answer.

Two methods don’t work: **linear walk** through 4,096-dim space (method 2, off-manifold) and **multi-layer Fourier rotation** (method 5, over-perturbation).

Big Idea

The fifteen-copy ensemble is not an obstacle to control — it’s an asset, if you know how to use it.

If you modify only one direction in only one layer (method 4), you have to fight fourteen other copies and they out-vote you. If you over-amplify across many layers (method 5), you break the network entirely. But if you inject a full centroid that captures *all fifteen copies’ simultaneous Friday signature at once* (method 1), the entire ensemble cooperates. The centroid $M[\text{Friday}]$ is, in some sense, the **fingerprint of every counting circuit aligned on Friday simultaneously**. You don’t need to find the circuits one by one. You only need their joint fingerprint.

The deeper point

You might expect that controlling a system with many redundant components requires finding and adjusting each component. Llama suggests something different. The redundant ensemble has a property we didn't anticipate: **the centroid is a summary statistic over all of it**. By injecting one summary statistic, we coordinate all the components without ever identifying them individually.

This is actually a hopeful finding for interpretability and control work. We don't need to map every circuit in the model in order to influence it. We just need to know what its activation looks like, on average, when it's about to do the thing we want — and dictate that average back in.

Caveat: this works only for things the model can already do (where we have many examples to average). For new operations the model can't perform, no centroid exists, and dictation has nothing to inject.

What this means for the broader research: the kind of control that works on Llama right now is *not* the same as the kind that worked on the toy in Book 1. The toy admitted clean walks at any small step ($\Delta = \pm 1, \pm 5, \pm 12$) on every direction we tried. Llama's control surface is more like a few well-worn paths between known states, plus a sharp drop into incoherence if you push too hard. Real LLMs are observable; they are partially manipulable; they are not yet *controllable* in any engineering-grade sense.

Closing Thoughts

In book one we cracked open a single tiny brain and found twelve beautiful circles — one task, one network, one algorithm, twelve hands rotating at twelve speeds. It was a self-contained jewel.

In this book we tried the same trick on a real giant LLM and found six beautiful dials, at one specific layer, running the calculator for an entire range of counting tasks. Same trick, bigger discovery. The dials were *found* (not invented) by the model itself during training. They are simple, mathematically clean, and they generalize.

There’s something almost philosophical about all this. The Fourier algorithm — a way of decomposing complicated patterns into simple rotations — was discovered by Joseph Fourier in 1807 while he was studying how heat spreads through metal. He wrote the math down because it solved his equations. Two hundred and nineteen years later, an artificial intelligence trained on web text, with no human guidance, reinvented the same algorithm. Not because anyone told it to. Because it works.

When you have a useful mathematical structure, it gets found over and over again, by very different minds, working on very different problems. Fourier rediscovered it. Llama rediscovered it. The next AI brain will too.

This is, in some sense, the most reassuring thing we know about the inner life of these models: **they reach for the same mathematical patterns we do.** The brain is alien. The math, less so.

Sources you can read next, if you want to go deeper:

The original Goodfire paper, “Arithmetic in the Wild: Llama Uses Base-10 Addition to Reason about Cyclic Concepts,” available at <https://arxiv.org/abs/2605.01148>. Their code is public at <https://github.com/goodfire-ai/arithmetic-wild>.

Goodfire’s follow-up paper on “manifold steering,” which gets into the deeper geometry of how concepts are arranged in LLMs: <https://arxiv.org/abs/2605.05115>.

The original “grokking” modular arithmetic paper by Neel Nanda: <https://arxiv.org/abs/2301.05217>.

And, if you want to actually try this at home, you can run our replication notebook (free Colab + an A100 GPU runs the whole experiment in about five minutes). Ask your detective for the link.